

Core Knowledge Series: **Machine Learning**
Session Three: **SVM, Hierarchical Clustering,
Random Forest/Decision Trees**

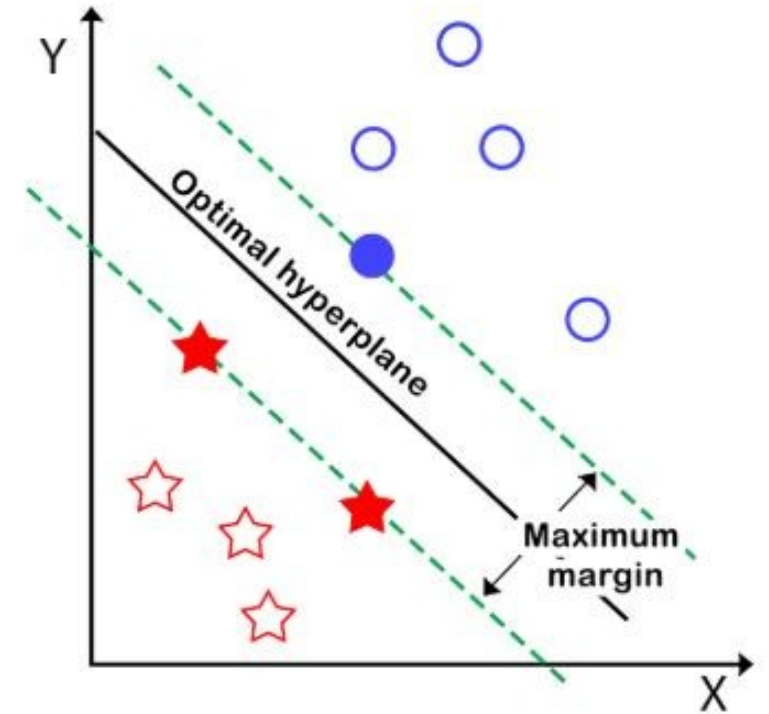
Bioinformatics Core (BSR)



Support Vector Machines

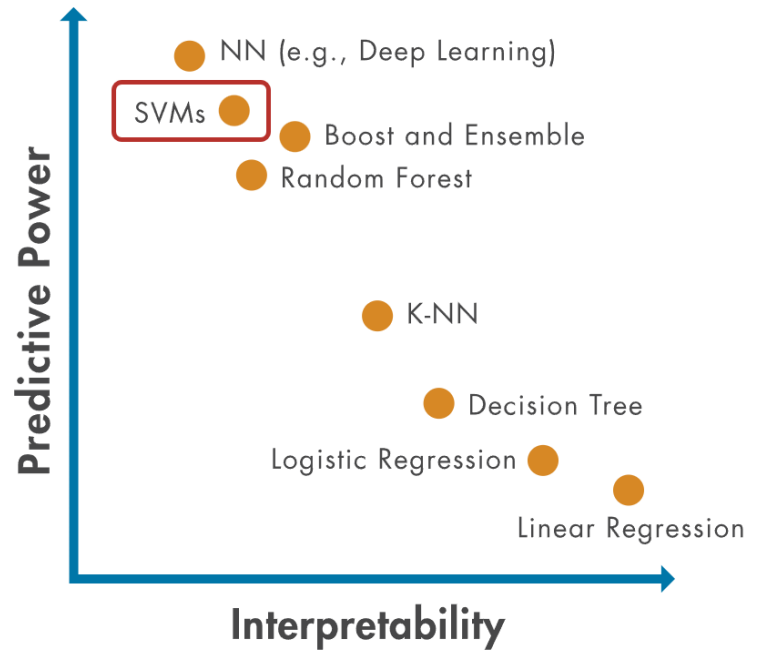
Support Vector Machines

- SVM is a supervised ML algorithm used primarily for classification
- Finds the best boundary—called a hyperplane—that separates different classes of data in a way that maximizes the distance (margin) between them.
- Real world example: Distinguishing between malignant and benign tumors



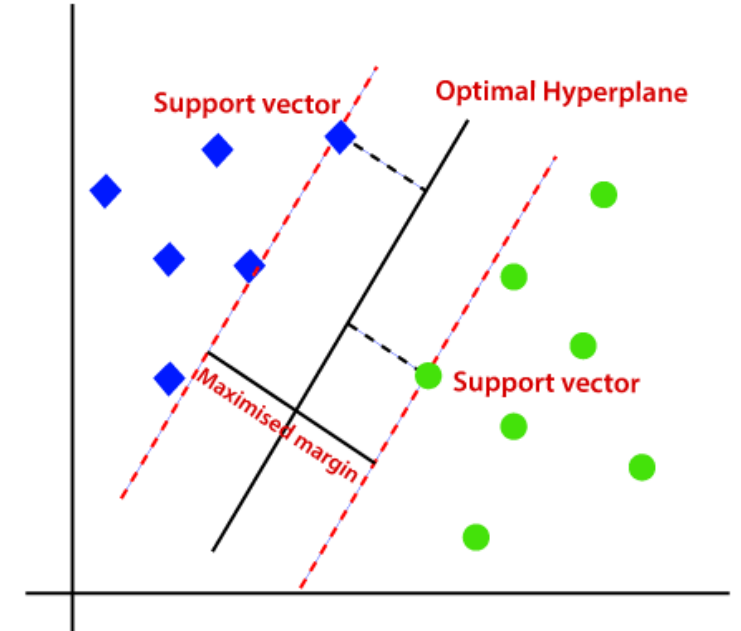
Support Vector Machines – Pros and Cons

Pros	Cons
Effective in high-dimensional spaces	Doesn't scale well to very large datasets
Can handle non-linear decision boundaries/data	Kernel and hyperparameter tuning (C, kernel choice) can be challenging
Memory efficient (only rely on a subset of training points - support vectors)	Outputs are not naturally probabilistic
Tend to generalize well, resisting overfitting	Harder to interpret than other models



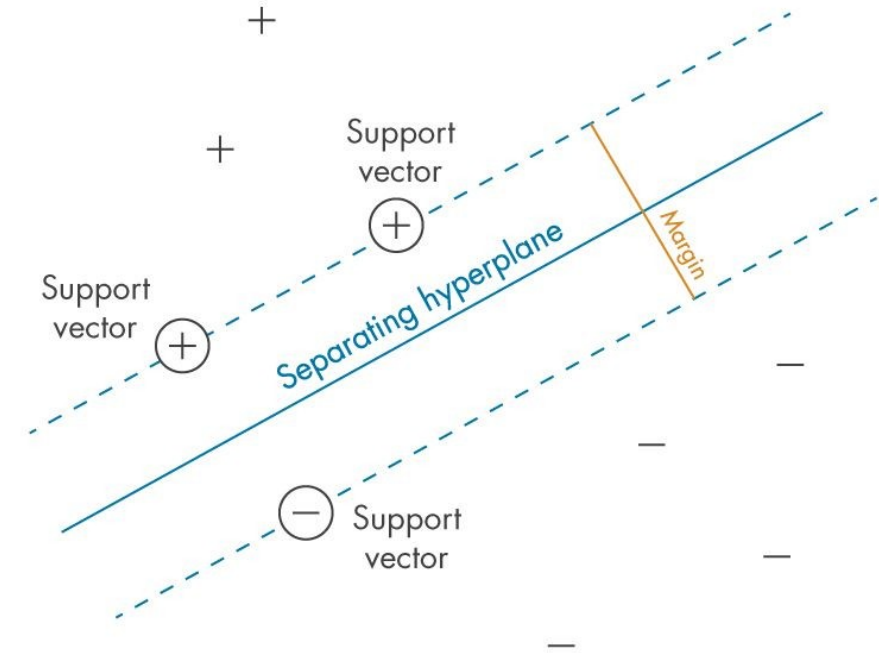
How do SVMs work?

- **SVM** searches for a boundary that best separates the data
- SVM selects the one with the **largest distance** from the nearest points of each class.
- **Larger margin** leaves more room for variation in new data
 - This reduces the chance that small differences will lead to misclassification.



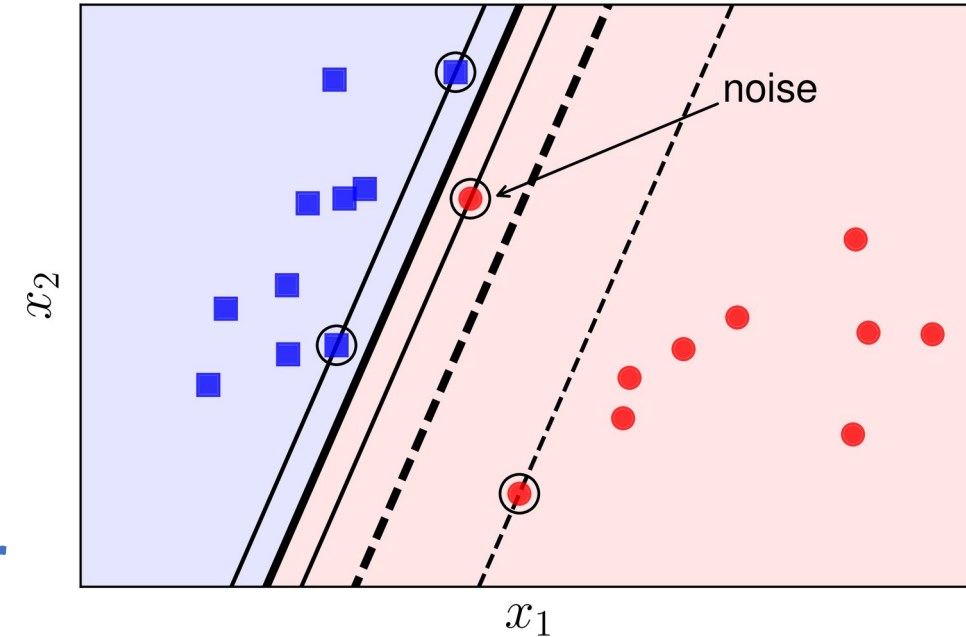
Support Vectors

- **Support vectors** are the data points closest to the initial decision boundary.
- **Final decision boundary** is decided using **only** support vectors
- Moving other points has little/no effect on the boundary.
- But what happens when the data cannot be perfectly separated by a boundary?



Soft Margin SVM

- Real-world data often contains **noise** and outliers
- **Soft margin SVM** allows some points to fall inside the margin or be misclassified
 - Maintains a wider, more generalizable boundary
- **Hard margin SVMs** tend to overfit the data and is strongly affected by outliers
- **Goal: Maximize margin while minimizing error**



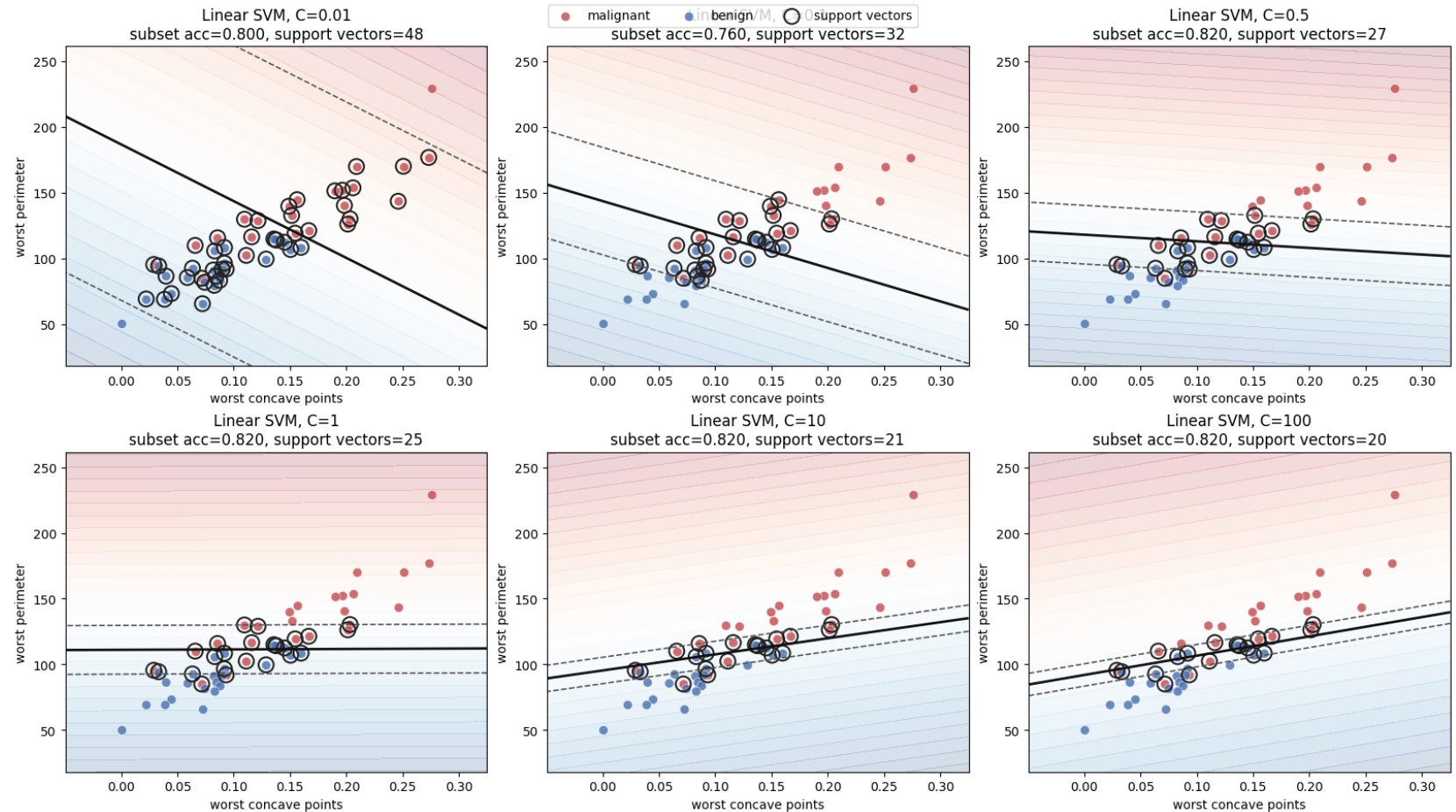
$$J(w, b) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w_i^x + b))$$

↑
Inverse of Margin

↑
Penalty for misclassifications

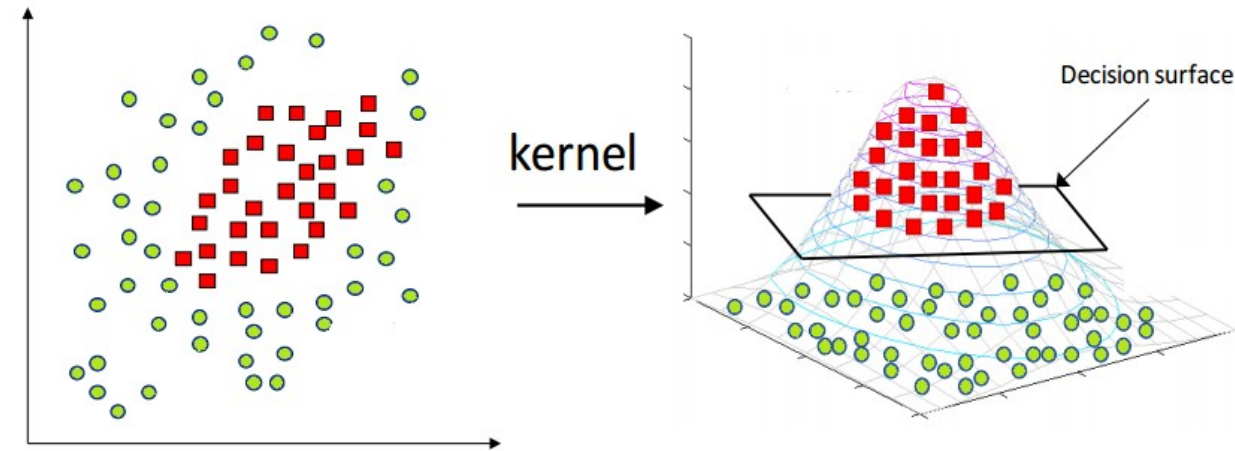
Affect of C Parameter Adjustment

- **C Parameter:** Controls how much the SVM penalizes classification errors.
 - High C: Fewer errors allowed (higher penalty), boundary tries harder to classify all points correctly.
 - Low C: More errors allowed (lower penalty), wider margin and less sensitivity to outliers.



Kernel Trick / Nonlinear SVM

- Not all datasets can be separated by a straight line
- Kernels transform data into a higher-dimensional space
- This allows SVMs to find nonlinear decision boundaries

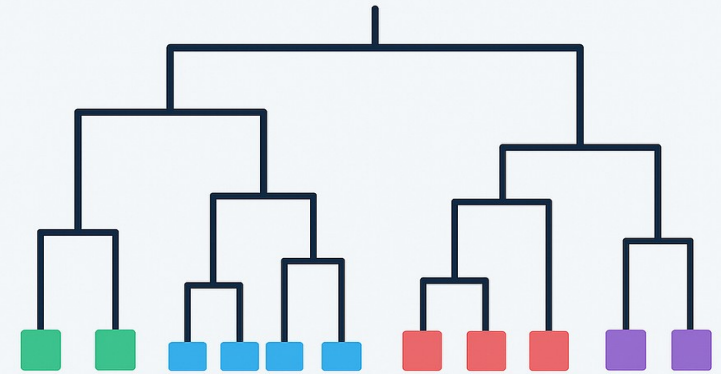


Hierarchical Clustering

Hierarchical Clustering

- **Unsupervised** machine learning method
- Groups similar observations into **clusters**
- Builds a **hierarchy of relationships** between observations
- Results are visualized in a **dendrogram (tree)**

HIERARCHICAL CLUSTERING THE TREE BEHIND YOUR DATA

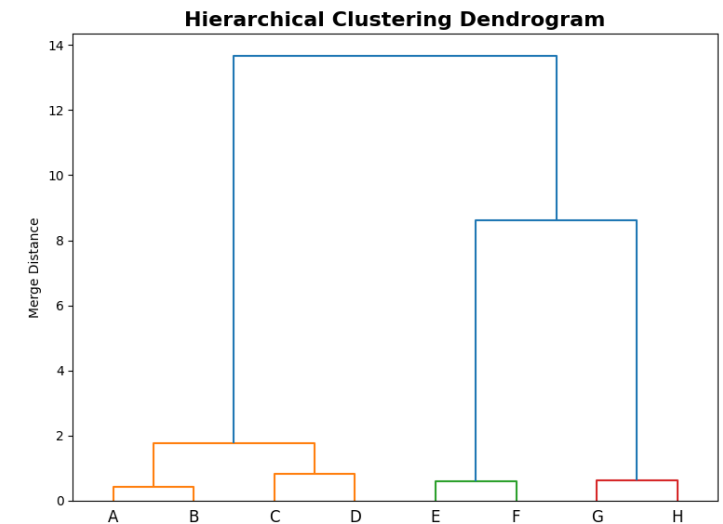
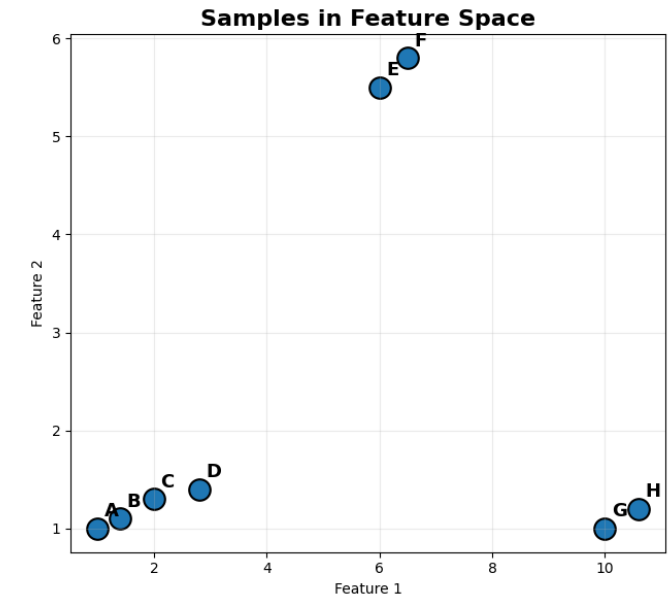


Hierarchical Clustering– Pros and Cons

Pros	Cons
Don't need to specify number of clusters	Computationally expensive for large datasets
Dendrogram is intuitive and interpretable	Sensitive to hyperparameters chosen
Reveals relationships between both samples and clusters	Heavily affected by noise and outliers
Commonly used for gene expression and genomic data	No objective “best” number of clusters is reported

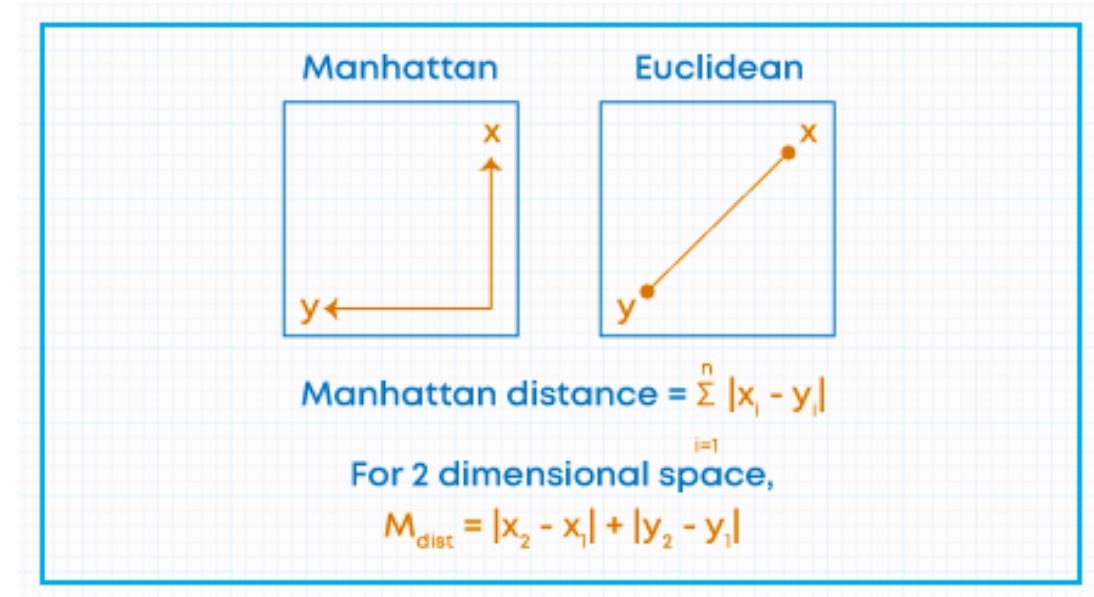
Hierarchical Clustering – How it works

- Start with each observation as its own cluster
 - Every sample begins in a separate cluster.
- Calculate distances between all clusters
 - Similarity is measured using a distance metric (e.g., Euclidean distance).
- Merge the two closest clusters
 - The most similar clusters are combined into a larger cluster.
- Repeat until all observations are grouped
 - Distances are recalculated after each merge until a single cluster remains.



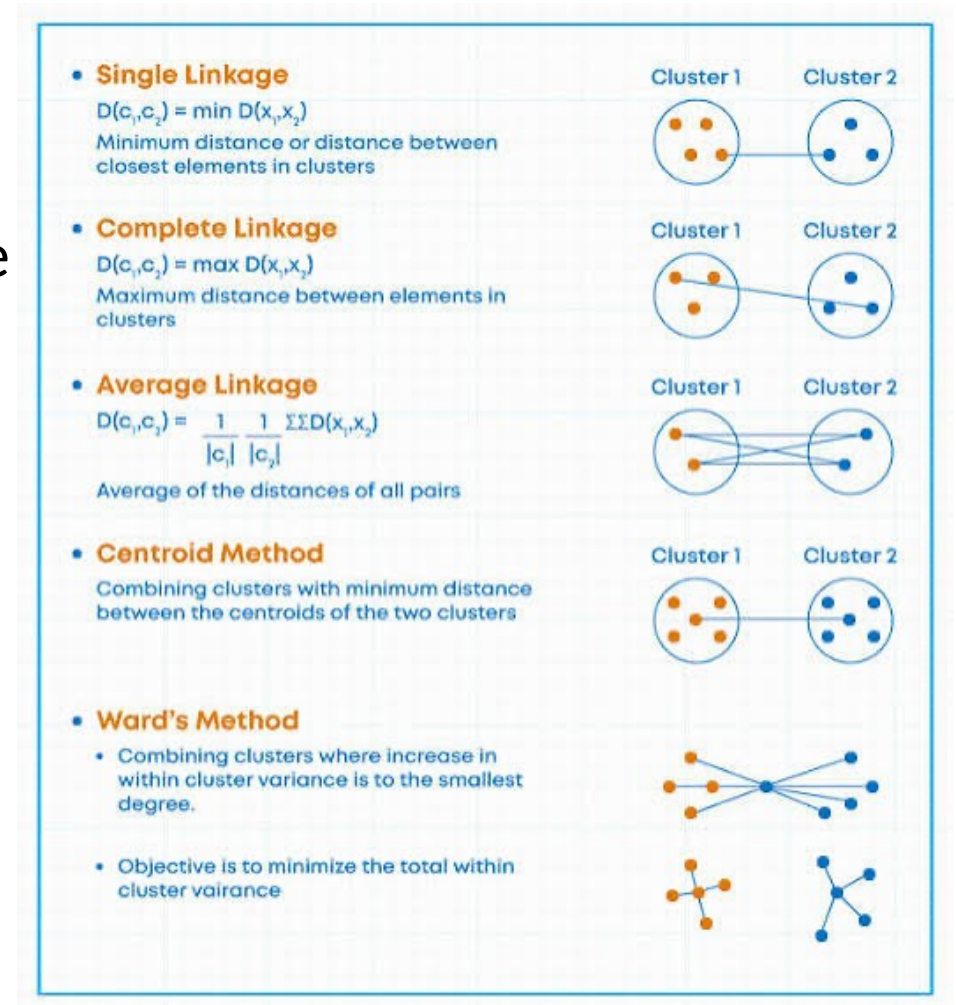
Distance Calculation Methods

- Euclidian Distance
 - Measures the straight-line distance between two measurements
- Manhattan Distance
 - Measures distance traveled across a grid
- Other distance metrics
 - Correlation Distance (Patterns)
 - Cosine Distance (Similarity in direction)



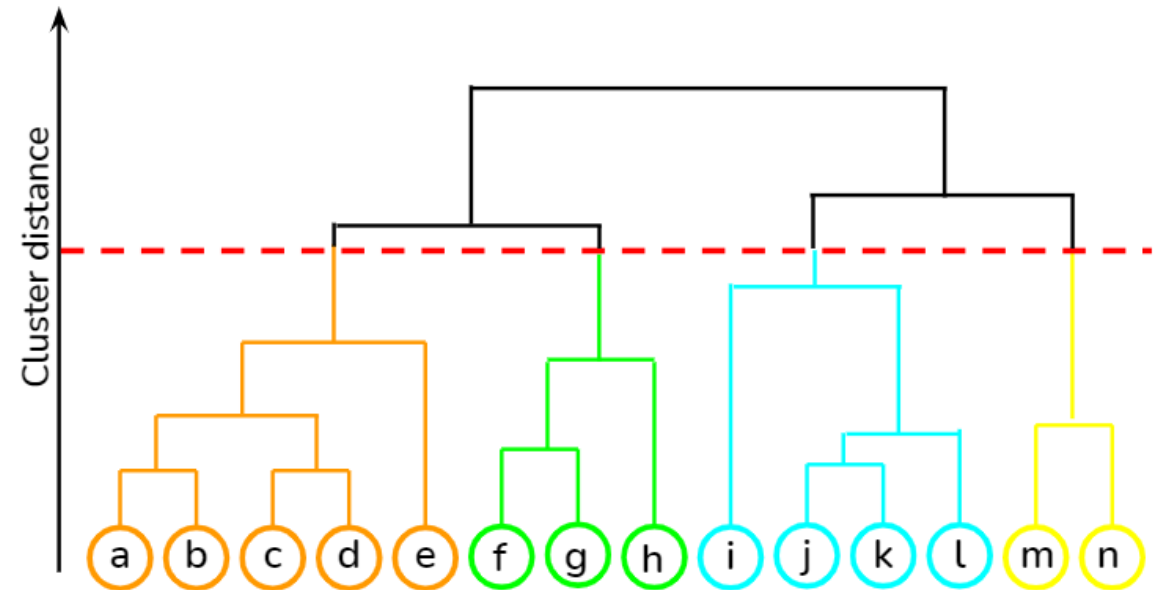
Linkage

- **Linkage** determines how the distance between clusters is calculated.
 - When clusters contain multiple samples, there is no single distance between them.
- **Single Linkage**
 - Uses the closest points between clusters.
- **Complete Linkage**
 - Uses the farthest points between clusters.
- **Average Linkage**
 - Uses the average distance between all points.
- **Ward's Linkage**
 - Merges clusters that result in the smallest increase in cluster variance.



Determining Clusters from Dendrogram

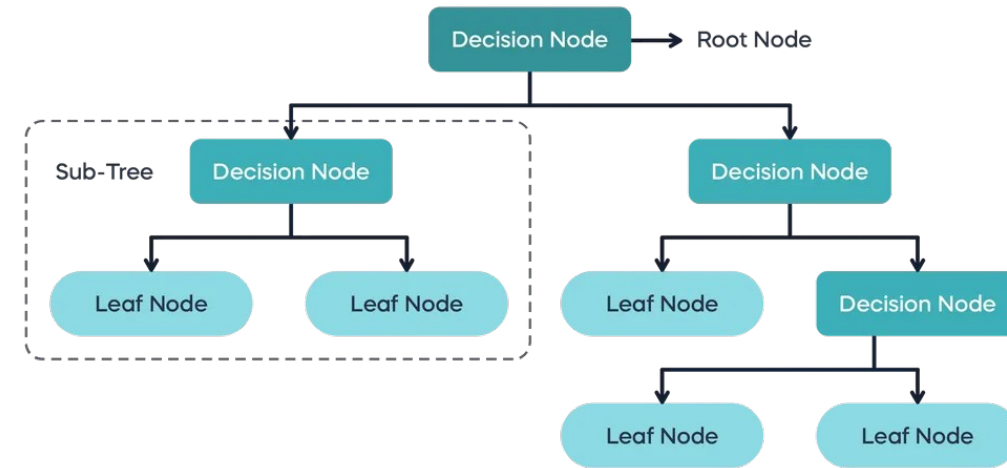
- Hierarchical clustering does not require a predefined number of clusters.
- Clusters are determined by selecting a **cut height** on the dendrogram.
- **Effect of Cut Height**
 - **Higher cut:** Fewer, larger clusters
 - **Lower cut:** More, smaller clusters



Decision Tree

Decision Tree

- Supervised machine learning method
- Uses a series of decision rules to make predictions
- Can be used for classification and regression
- Easy to interpret and visualize

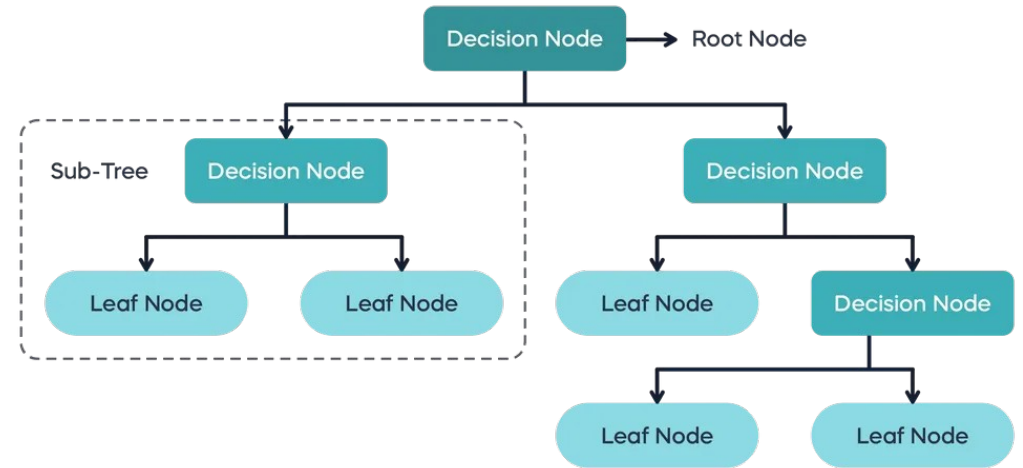


Decision Tree – Pros and Cons

Pros	Cons
Easy to interpret and explain	Prone to overfitting
Handles numeric and categorical features	Small data changes can cause large structural alterations in the tree
No feature scaling required	Biased toward dominant features
Provides easy to follow decision trees	Often less accurate than ensemble methods

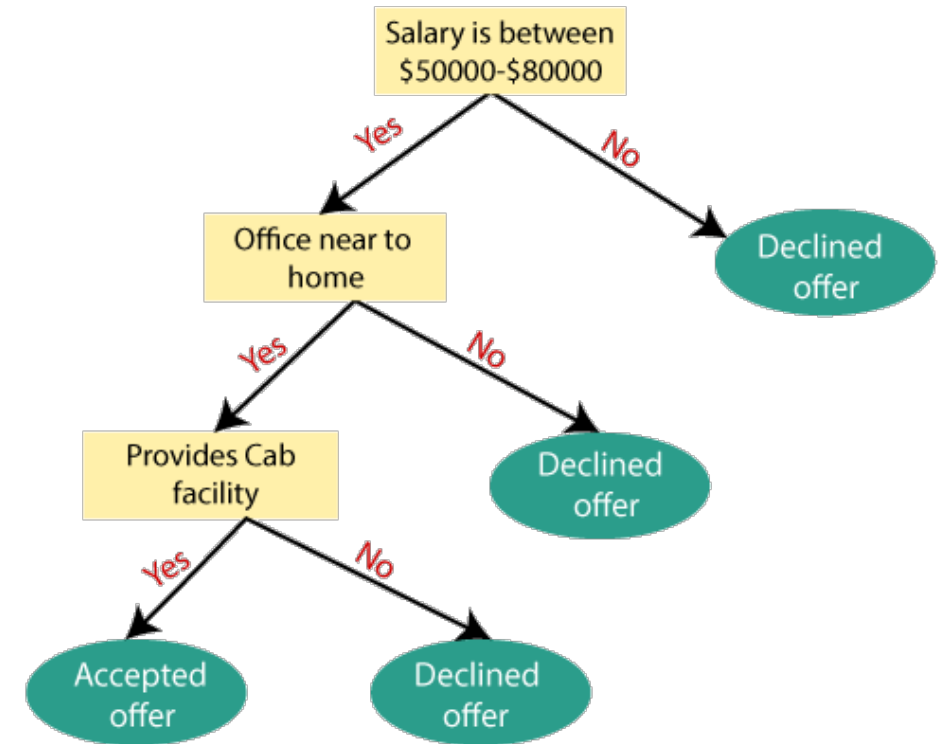
Decision Tree Structure

- **Root Node**
 - First decision in the tree
- **Internal Nodes**
 - Intermediate decision nodes
- **Branches**
 - Outcomes of decision rules
 - Connect parent and child nodes
- **Leaf Nodes**
 - Terminal nodes that make the final prediction
 - Take majority class as prediction



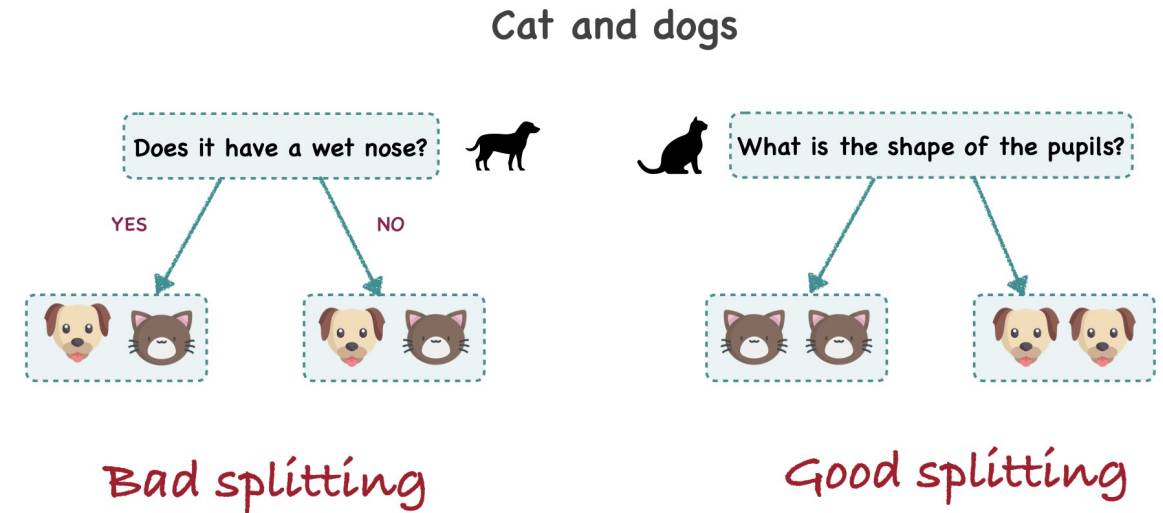
How do Decision Trees work?

- Start with all observations in one group.
- Find the best feature and threshold to cleanly separate the data.
- Partition observations into child nodes based on the split rule and recursively evaluate additional splits.
- Stop when a stopping criterion is reached and assign a prediction



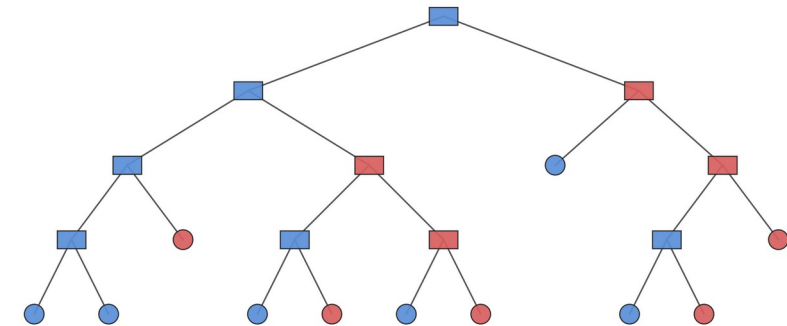
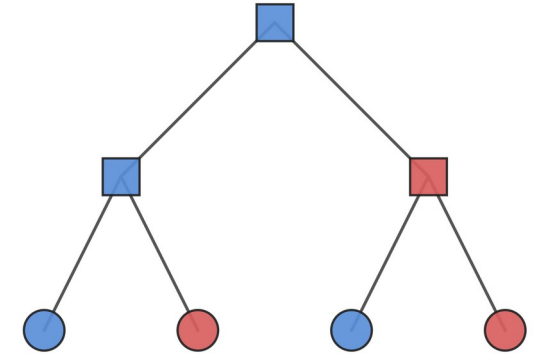
Choosing the best split at decision nodes

- Multiple features and thresholds are evaluated at each node.
- The **best split** is the one that separates the classes most effectively.
- **Gini Impurity** is the most common method used to evaluate the quality of a split.
 - Measures how mixed the classes are within a node.



Controlling Tree Growth

- Decision trees **continue splitting** until a **stopping criterion** is reached.
- **Limiting tree growth** can **reduce overfitting** and **improve generalization**.
- Several hyperparameters control when the tree stops splitting.
 - **Max_depth**: Maximum number of levels in the tree
 - **Min_samples_split**: Minimum number of observations required to split a node.
 - **Min_samples_leaf**: Minimum number of observations required in a leaf node.



Limitations of a single decision tree

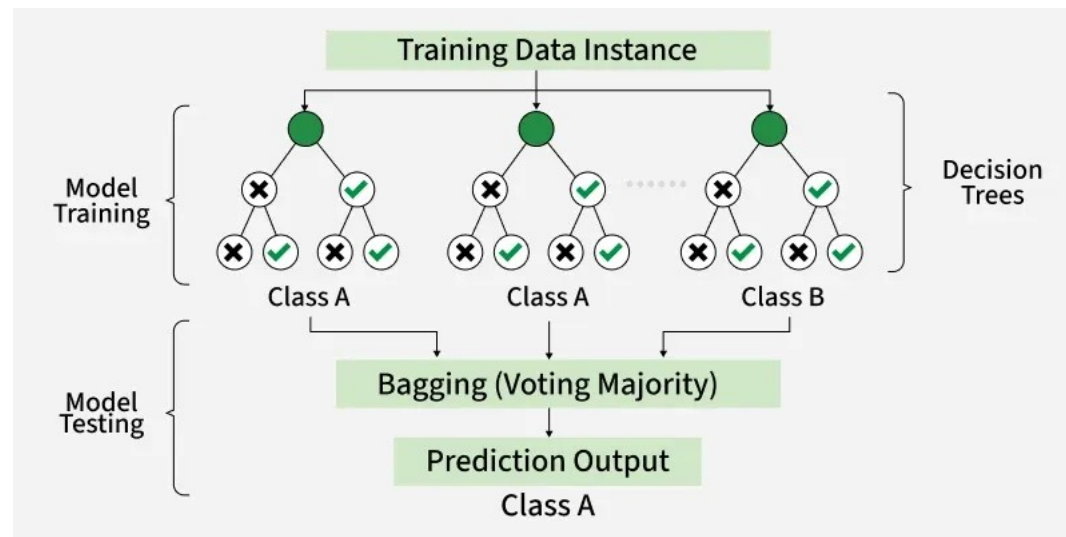
- Decision trees are **highly sensitive** to the training data.
- Small changes in the data can produce very different trees.
- **Deep trees** may **overfit** and memorize training examples.
- **Predictions** from a single tree can be **unstable**.
- **How can we maintain the interpretability of decision trees while reducing overfitting and improving prediction accuracy?**



Random Forest

Random Forest

- A Random Forest is a collection of decision trees that work together to make predictions.
- Each tree is trained on a slightly different dataset.
- Predictions from all trees are combined.
- Reduces overfitting and improves model stability.

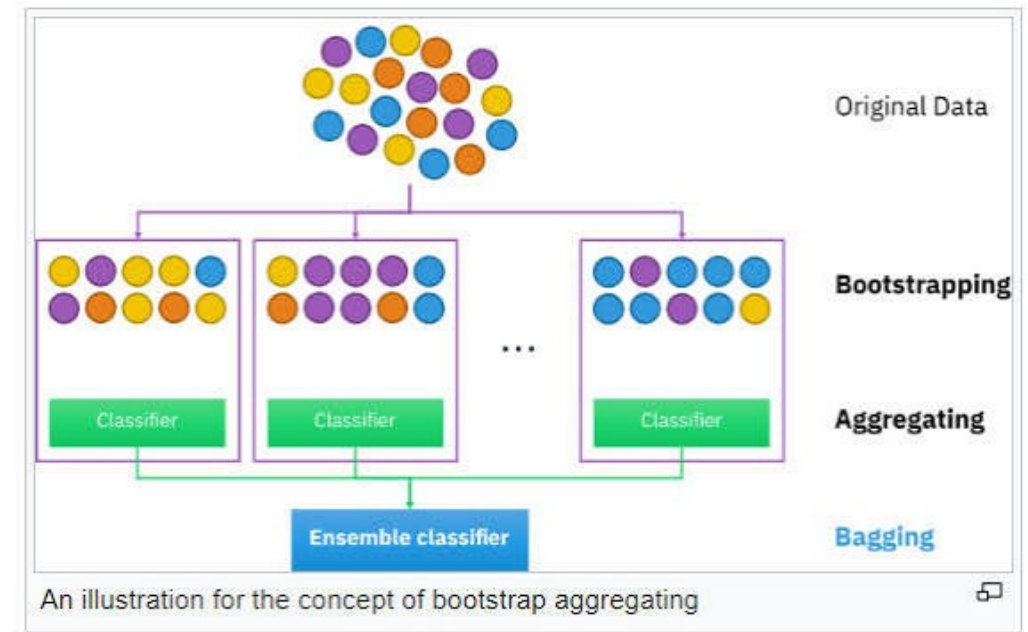


Random Forest– Pros and Cons

Pros	Cons
More accurate than a single decision tree	Less interpretable than a single decision tree
Less prone to overfitting	More computationally expensive
Robust to noise and outliers	Larger memory footprint
Handles high-dimensional data well	Requires more hyperparameter tuning

How Random Forest Works

- Create multiple random training datasets by **sampling observations with replacement**
- **Train** a separate **decision tree** on each **bootstrapped dataset**
- At each split, evaluate only a **random subset of features**.
 - Different trees learn different patterns
- Combine predictions from all trees
 - **Majority vote**



How many trees to build in the forest?

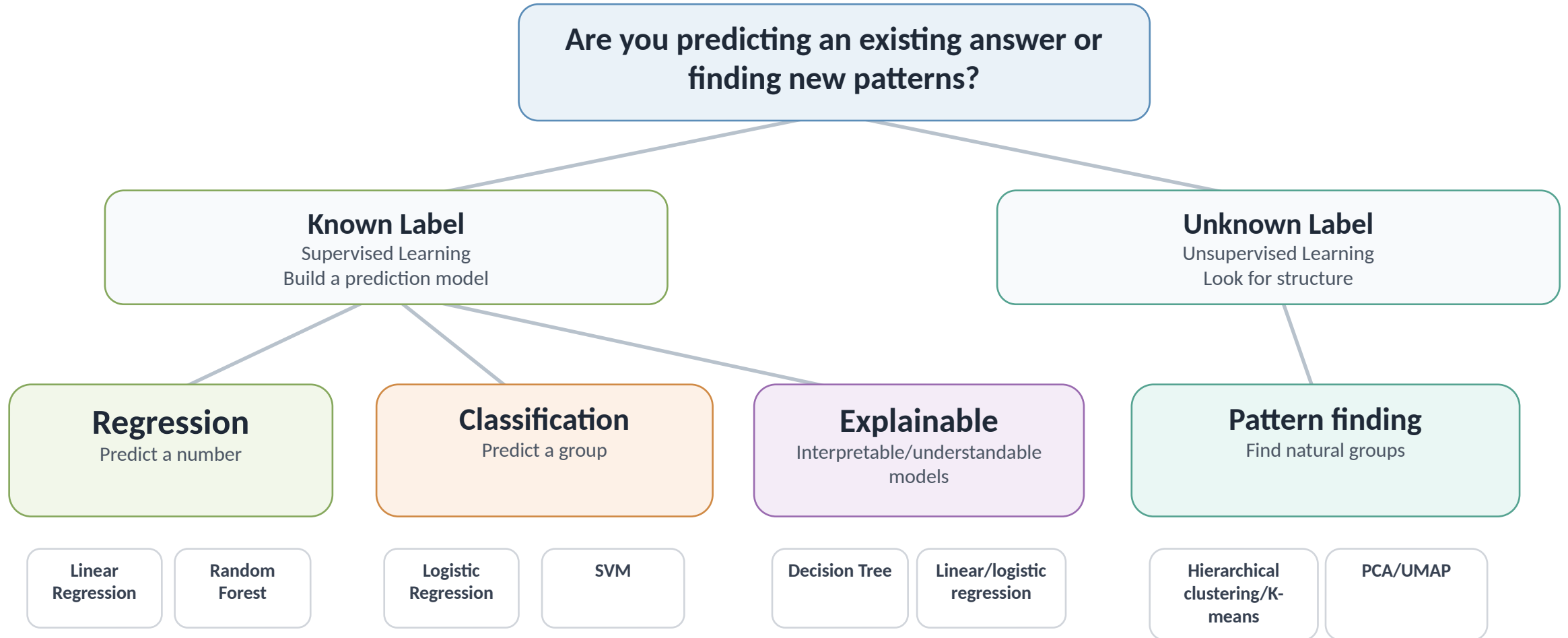
- **Number of Trees (`n_estimators`)**

- Controls how many decision trees are included in the forest.
- More trees generally improve model stability and performance.
- Performance gains eventually plateau.
- More trees increase training time and memory usage.

- **Number of random features (`max_features`)**

- Smaller values increase tree diversity
- Larger values make trees more similar
- Typically set to the square root of the total number of features

Choosing a machine learning model



Additional Models Not Covered

- **Gradient Boosting:**

- Combines many simple models to make stronger predictions
- Each model fixes the mistakes of the previous ones

- **Neural Networks:**

- Learns complex patterns from large datasets using artificial neurons

- **Deep Learning:**

- Uses larger neural networks for complex data



**Please scan the QR
code to take our 1-
minute survey!**

**Your feedback
ensures we provide
you with the best
content**

